Singleton Pattern Implementation Analysis

Experimental Results

Software Design Pattern Study

April 22, 2026

1 Instance Validation Results

1.1 Evidence from Object IDs

All implementations maintain a single instance. Same object ID returned on multiple calls:

Table 1: Instance Validation - Object IDs

Implementation	Call 1	Call 2	Result
Eager	4416955328	4416955328	
Lazy	4421460928	4421460928	
Synchronized	4416961040	4416961040	
Double-Checked	4421459920	4421459920	
Bill Pugh	4421461264	4421461264	
Enum	4421460256	4421460256	

Conclusion: All implementations pass instance validation.

2 Multithreading Results

2.1 Concurrent Access from 20 Threads

Test: 20 concurrent threads requesting singleton instances.

Table 2: Threading Test - Concurrent Access

Implementation	Unique Objects	Unique Clients	Result
Eager	1	1	PASS
Lazy	20	20	FAIL
Synchronized	1	1	PASS
Double-Checked	1	1	PASS
Bill Pugh	1	1	PASS
Enum	1	1	PASS

Note: Lazy is not thread-safe. In concurrent access, it produced multiple instances and multiple clients.

2.2 Why Lazy Is Not Thread-Safe

The lazy singleton uses a check-then-create pattern. If two threads enter `__new__` at the same time, both can see `_instance is None` before either one assigns the object. That creates a race condition.

```
class LazySingleton:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            # Race window: two threads can both pass this check
            cls._instance = super().__new__(cls)
        return cls._instance
```

Without a lock around the creation block, the implementation is not thread-safe by design.

Conclusion: Lazy failed the concurrent load test, while the other implementations maintained a single instance.

3 Performance Measurements

3.1 Initialization and Access Times

Table 3: Performance Comparison

Implementation	First Access (s)	Avg Access (µs)
Eager	0.00000000	0.0390
Bill Pugh	0.00000095	0.0354
Enum	0.00000095	0.0282
Double-Checked	0.00000095	0.0679
Lazy	0.00000095	0.1397
Synchronized	0.00000167	0.1523

3.2 Performance Rankings

Fastest (Repeated Access):

1. Enum Singleton: 0.0282 µs
2. Bill Pugh Singleton: 0.0354 µs
3. Eager Singleton: 0.0390 µs

Slowest (Repeated Access):

1. Double-Checked Locking: 0.0679 µs
2. Lazy Singleton: 0.1397 µs
3. Synchronized Singleton: 0.1523 µs

Performance Variance: Synchronized is 5.4× slower than Enum.

4 Comparative Summary

4.1 Key Findings

Table 4: Implementation Comparison

Implementation	Instance	Thread-Safe	Performance	Overall
Eager			Good	Excellent
Bill Pugh			Excellent	Excellent
Enum			Excellent	Excellent
Double-Checked			Moderate	Good
Lazy			Slow	Fair
Synchronized			Slowest	Poor

4.2 Observations

- **Instance Validation:** All 6 implementations maintain single instance (100% success)
- **Thread Safety:** 5 implementations are thread-safe by design; Lazy is not thread-safe and failed under concurrency
- **Which are fastest:** Enum, Bill Pugh, and Eager
- **Which are thread-safe in practice:** Eager, Synchronized, Double-Checked, Bill Pugh, and Enum
- **Which show inconsistencies:** None detected

4.3 Recommendations

- **Best Overall:** Enum or Bill Pugh (fast, thread-safe by design)
- **Balanced:** Eager Singleton (simple, good performance)
- **Avoid:** Lazy (not thread-safe) and Synchronized (poor performance)

5 Conclusion

All six Singleton implementations maintain single instances in single-threaded validation. Under concurrency, Lazy is not thread-safe and failed the thread test. Performance varies significantly, with Enum and Bill Pugh offering optimal throughput (0.0282–0.0354 µs per access).